National University of Computer & Emerging Sciences, Karachi
Computer Science Department
Spring 2026, Lab Manual – 05

Course Code: CS-1004	Course: Object Oriented Programming Lab
Instructor :	Muhammad Owais

LAB - 5

Static Keyword & has-a Relationship

Static Keyword

We can apply static keyword with variables, methods, blocks and nested classes. The static keyword belongs to the class rather than an instance of the class.

Static Variable

The static variable can be used to refer to the common property of all objects (which is not unique for each object), for example, the company name of employees, college name of students, etc. The static variable gets memory only once in the class area at the time of class loading.

Example

```
public class Test{
    public static void main(String args[]){
        Student s1 = new Student( r: 111, n: "Ali");
        Student s2 = new Student( r: 222, n: "Abid");
        s1.display();
        s2.display();
    }
}
class Student{
    int rollno;//instance variable
    String name;
    static String college ="IT College";//static variable
    //constructor
    Student(int r, String n){
        rollno = r;
        name = n;
    }
    //method to display the values
    void display (){System.out.println(rollno+" "+name+" "+college);}
}
```

Static Methods

A static method belongs to the class rather than the object of a class. A static method can be invoked without the need for creating an instance of a class. It can access static data member and can change the value of it.

Example

```java
public class Test{
    public static void main(String args[]){
        Student.change();
        //creating objects
        Student s1 = new Student( r: 111, n: "Ali");
        Student s2 = new Student( r: 222, n: "Abid");
        Student s3 = new Student( r: 333, n: "Bilal");
        //calling display method
        s1.display();
        s2.display();
        s3.display();
    }
}
class Student{
    int rollno;
    String name;
    static String college = "New Age College";
    static void change(){
        college = "IT College";
    }
    Student(int r, String n){
        rollno = r;
        name = n;
    }
    void display(){System.out.println(rollno+" "+name+" "+college);}
}
```

Has-a Relationship / Association

A Has-A relationship in Java means that one class contains an object of another class as its data member. It shows a relationship where one object uses or owns another object. In simple words, if one class needs another class to complete its work, we say it has a Has-A relationship. It is mainly achieved by object creation inside another class.

For example, a Car has an Engine. A car cannot run without an engine, so the Car class contains an object of the Engine class.

There are two types:

1. Aggregation (Weak Has-A)
2. Composition (Strong Has-A)

Aggregation (Weak Has-A Relationship)

Aggregation is a weak type of Has-A relationship in Java where one class contains another class object, but the contained object can exist independently. The life cycle of both objects is separate, meaning if the main object is destroyed, the contained object can still exist. It represents a general "has-a" relationship rather than a strong part-of relationship.

Example:

A school has teachers, but teachers can work in different schools and are not dependent on one specific school. In programming, the object is usually created outside the class and passed to it, which shows loose connection and weak dependency.

```java
class Teacher {
    String name;

    Teacher(String name) {
        this.name = name;
    }
}

class School {
    private Teacher teacher;   // Aggregation

    School(Teacher teacher) {
        this.teacher = teacher;   // Object passed from outside
    }

    void display() {
        System.out.println("Teacher: " + teacher.name);
    }
}

public class Main {
    public static void main(String[] args) {
        Teacher t1 = new Teacher("Ali");   // Teacher created
            independently
        School s1 = new School(t1);
        s1.display();
    }
}
```

Composition (Strong Has-A Relationship)

Composition is a strong type of Has-A relationship in Java where one object completely owns another object. The contained object cannot exist independently without the main object, and both share the same life cycle. If the main object is destroyed, the contained object is also destroyed. It represents a strong "part-of" relationship.

Example:

A car has an engine; the engine is an essential part of the car and does not meaningfully exist without it in that context. In programming, the contained object is usually created inside the main class, which shows strong dependency and ownership.

```java
class Engine {
    void start() {
        System.out.println("Engine starts");
    }
}

class Car {
    private Engine engine;   // Composition

    Car() {
        engine = new Engine();   // Engine created inside Car
    }

    void startCar() {
        engine.start();
        System.out.println("Car starts");
    }
}

public class Main {
    public static void main(String[] args) {
        Car c = new Car();
        c.startCar();
    }
}
```